
DisrtuCare

Distributeur de pilules intelligent

Rapport d'analyse technique

Fonctionnement et pipeline matériel & logiciel

Projet Application compagnon + firmware embarqué

Plateformes Android / iOS (Expo) + NodeMCU ESP8266

Stack React Native, TypeScript, SQLite, Arduino C++

Date 25 mai 2026

Table des matières

1	Vue d'ensemble	2
2	Architecture globale	2
3	Le pipeline matériel (firmware)	2
3.1	Composants et câblage	3
3.2	Le moteur — cœur de la distribution	3
3.3	La gestion du temps (astuce intelligente)	3
3.4	La boucle principale (<code>distruccare.ino</code>)	4
3.5	Persistance EEPROM	4
4	Le protocole de communication (HTTP/JSON)	4
4.1	Application → Appareil	4
4.2	Appareil → Application (par sondage)	5
5	Le pipeline logiciel (application mobile)	5
5.1	Stack technique	5
5.2	La base de données SQLite	5
5.3	La couche matériel (<code>httpService.ts</code>)	5
5.4	La boucle de rétroaction complète (pipeline bout en bout)	5
5.5	Les écrans	6
5.6	Notifications locales	6
5.7	Design & accessibilité	6
6	État d'avancement	7
7	Points forts et observations	7

1. Vue d'ensemble

DisrtuCare est un système de **distributeur de pilules intelligent** destiné principalement aux **personnes âgées**. Il combine deux mondes :

- **Un matériel autonome** (à base de NodeMCU/ESP8266) qui distribue physiquement les pilules aux heures programmées.
- **Une application mobile compagnon** (Expo / React Native) qui sert de tableau de bord, de suivi d'observance et de télécommande.

Le projet a connu une **évolution importante** visible dans le code :

- Le cahier des charges initial (`rules.md`) interdisait toute connectivité (« simuler l'interaction, pas de Bluetooth/WiFi/cloud »).
- Cette contrainte a été **levée** : le projet utilise désormais une vraie communication **WiFi/HTTP**. On voit même les vestiges d'une phase Bluetooth abandonnée (`bleService.ts` ne fait plus que ré-exporter le service HTTP).

Incohérence documentaire : `rules.md` mentionne un *Arduino Uno* alors que le matériel réel est un **NodeMCU V3 (ESP8266)** — choisi car il intègre le WiFi nativement.

2. Architecture globale

```
+-----+
| Telephone (Expo app) | <- Tableau de bord, historique, réglages
| React Native + SQLite |
+-----+
      | HTTP/JSON sur le meme reseau WiFi (LAN)
      | (polling toutes les 3 s + commandes POST)
      v
+-----+
| NodeMCU V3 (ESP8266) | <- Serveur HTTP sur le port 80
| Firmware Arduino |
+-----+
      | I2C | GPIO | GPIO
      v v v
LCD 16x2 Moteur pas LED bleue
+ DS3231 a pas + Bouton
(RTC) 28BYJ-48 poussoir
      via ULN2003
```

Le point essentiel : **le matériel fonctionne de façon autonome**. Même sans téléphone connecté, le distributeur connaît l'heure (RTC), distribue aux heures programmées (sauvegardées en EEPROM), et confirme la prise via le bouton. L'application **enrichit** l'expérience mais n'est pas indispensable au fonctionnement de base.

3. Le pipeline matériel (firmware)

3.1 Composants et câblage

Composant	Rôle	Connexion NodeMCU
NodeMCU V3 (ESP8266)	Cerveau + serveur WiFi	—
LCD 16x2 (I ² C, PCF8574)	Affichage état/heure/IP	D1 (SCL), D2 (SDA)
RTC DS3231	Horloge temps réel (pile CR2032)	Même bus I ² C
Moteur 28BYJ-48 + ULN2003	Rotation du carrousel	D5-D8 → IN1-IN4
LED bleue	Alerte visuelle (clignotement)	D0
Bouton poussoir	Confirmation de la prise	D3 (INPUT_PULLUP)

Détails techniques relevés dans `config.h` :

- **D8 (GPIO15)** doit être LOW au démarrage → le firmware appelle `motorStop()` au boot pour ne pas empêcher le démarrage de l'ESP8266.
- **D3 (GPIO0)** ne doit pas être maintenu enfoncé au boot (sinon mode flash).
- **D0 (GPIO16)** est en sortie uniquement (pas d'INPUT_PULLUP possible).

3.2 Le moteur — cœur de la distribution

Dans `motor.cpp`, le 28BYJ-48 est piloté en **demi-pas** (half-step, séquence de 8 micro-pas) pour plus de couple et de fluidité :

- 2048 pas complets/tour → 4096 en demi-pas.
- **4 compartiments** → `STEPS_PER_SLOT = 1024` demi-pas par slot, soit ~2 secondes de rotation par compartiment (à 2 ms/pas).
- Après chaque rotation, `motorStop()` coupe le courant des bobines (évite la surchauffe).

3.3 La gestion du temps (astuce intelligente)

`rtc_time.cpp` gère **deux sources de temps** :

1. **DS3231 matériel** s'il est détecté sur le bus I²C → garde l'heure même après coupure de courant.
2. **Horloge logicielle de secours** si aucun RTC n'est câblé : amorcée par l'app via `/settime` et avancée avec `millis()`. Elle dérive un peu et se réinitialise au reboot, mais l'app la re-synchronise à chaque connexion.

Le concept `rtcIsRunning()` empêche une distribution parasite à 00 :00 tant que l'heure n'est pas connue. **Cela rend le système fonctionnel même sans RTC câblé.**

3.4 La boucle principale (`distrucare.ino`)

Le `loop()` exécute 4 tâches en continu, sans blocage :

```
loop() ->
|- server.handleClient() // repond aux requetes HTTP de l'app
|- checkSchedule() // l'heure correspond-elle a AM ou PM ?
|- checkButton() // l'utilisateur a-t-il presse le bouton ?
+- updateLed() // clignotement non-bloquant de la LED
```

Cycle d'une distribution automatique :

1. `checkSchedule()` détecte que l'heure courante = heure AM (ou PM), et qu'on n'a pas déjà distribué cette minute (`lastDispensedMin`).
2. `dispense("AM")` : LCD affiche « Dispensing », le moteur tourne d'un slot, LCD affiche « Done ».
3. La **LED** se met à clignoter et le système passe en `awaitingConfirm = true`.
4. Un événement `{"evt":"dispensed","type":"AM"}` est mis en file (incrémente `eventId`).
5. L'utilisateur prend sa pilule et **appuie sur le bouton**.
6. `checkButton()` (anti-rebond 50 ms) éteint la LED, l'allume fixe 1,5 s en confirmation, puis émet `{"evt":"confirmed","type":"AM"}`.
7. À minuit, `lastDispensedMin` est réinitialisé pour les doses du lendemain.

3.5 Persistance EEPROM

L'horaire AM/PM et le nom du médicament sont stockés en EEPROM (flash simulée) avec un **octet magique 0xAB** validant l'initialisation des données. Ainsi, l'horaire **survit aux coupures de courant** sans dépendre de l'app.

4. Le protocole de communication (HTTP/JSON)

Le NodeMCU héberge un serveur HTTP sur le **port 80**. Toute la communication est en JSON, avec CORS activé.

4.1 Application → Appareil

Endpoint	Méth.	Action
<code>/status</code>	GET	Renvoie l'horaire, l'heure, le dernier événement
<code>/sync</code>	POST	Met à jour l'horaire + EEPROM + LCD
<code>/settime</code>	POST	Règle l'horloge DS3231
<code>/led</code>	POST	Allume/éteint la LED
<code>/motor</code>	POST	Fait tourner le distributeur (test)

4.2 Appareil → Application (par sondage)

L'app ne reçoit pas de push : elle **interroge /status toutes les 3 secondes**. La réponse contient un compteur `event_id` et un `last_event`. Quand `event_id` change, l'app sait qu'un nouvel événement matériel s'est produit et déclenche le gestionnaire correspondant.

```
{ "am":"08:00", "pm":"20:00", "med":"...", "time":"14:30",
  "rtc":true, "awaiting":false, "event_id":3,
  "last_event":{"evt":"confirmed","type":"AM"} }
```

C'est un modèle de **sondage par compteur d'événements** — simple et robuste sur un microcontrôleur, sans WebSockets.

5. Le pipeline logiciel (application mobile)

5.1 Stack technique

- **Expo ~54 / React Native 0.81 / React 19 / TypeScript**
- Navigation par onglets : **Dashboard / History / Device / Settings**
- **expo-sqlite** : base locale, hors-ligne par défaut
- **expo-notifications** : rappels locaux
- **react-native-reanimated** : animations fluides

5.2 La base de données SQLite

Deux tables :

- **schedule** : une seule ligne (médicament, heures AM/PM, notifications, IP de l'appareil, drapeau setup).
- **dose_logs** : journal des prises, contrainte `UNIQUE(date, dose_type)` → une entrée AM et une PM par jour, statut `taken/missed/late/pending`.

5.3 La couche matériel (`httpService.ts`)

Pont entre l'app et l'appareil, le singleton `HttpService` gère :

- **connect()** : ping `/status` (timeout 6 s), puis envoie automatiquement `/settime` pour synchroniser l'horloge de l'appareil sur celle du téléphone, puis démarre le sondage.
- **Sondage** : toutes les 3 s, récupère `/status`. Si l'`event_id` a changé, notifie les abonnés. En cas d'échec → déconnexion automatique.
- L'IP de l'appareil est **persistée en SQLite** → reconnexion sans ressaisie.

5.4 La boucle de rétroaction complète (pipeline bout en bout)

```
1. Reglages : l'utilisateur regle 08:00 / 20:00 dans SettingsScreen
   |
   |-> Ecrit en SQLite local
   |-> POST /sync -> l'appareil met à jour son EEPROM + LCD
   +-> scheduleDoseReminders() -> notifications locales programmées
```

2. A 08:00 : l'appareil distribue automatiquement (moteur + LED clignote)
+--> evenement "dispensed" place dans /status
3. L'app (en sondage) voit l'event_id changer -> l'affiche dans le journal
4. L'utilisateur prend la pilule et APPUIE sur le bouton physique
+--> evenement "confirmed" dans /status
5. DashboardScreen capte "confirmed" -> logDose('AM','taken')
|-> Ecrit "taken" en SQLite
+--> Affiche une alerte "Dose Confirmed"
6. Les statistiques d'observance (anneau 30 jours) se recalculent

Le geste physique sur le distributeur **enregistre automatiquement la prise dans l'app** — c'est le point fort de l'intégration (DashboardScreen.tsx).

5.5 Les écrans

Écran	Rôle
Dashboard	Prochaine dose (compte à rebours), anneau d'observance 30 j, cartes AM/PM (Taken/Missed/Late), auto-log depuis le matériel
History	Journaux quotidiens/hebdomadaires, codes couleur
Device	Saisie de l'IP, connexion, contrôles de test (Sync, LED, moteur), journal d'événements en direct
Settings	Heures AM/PM, nom du médicament, toggle notifications, reset des logs du jour

5.6 Notifications locales

À chaque sauvegarde de l'horaire, l'app annule puis reprogramme **4 notifications quotidiennes** : rappel AM, rappel PM, relance AM +30 min, relance PM +30 min. Tout est local, sans internet.

5.7 Design & accessibilité

Conforme au cahier des charges « personnes âgées » : grandes polices (18–22 px), fort contraste (fond bleu marine #0F172A, accent turquoise #14B8A6), grandes cibles tactiles (≥ 56 px), code couleur **vert = pris** / **jaune = en retard** / **rouge = manqué**, animations douces.

6. État d'avancement

Phase	Statut	Notes
1 — Câblage physique	✗	Moteur D5-D8, LED, bouton non câblés
2 — Protocole HTTP	OK	Défini et implémenté
3 — Firmware	OK	Téléversé, WiFi connecté, endpoints vérifiés
4 — Intégration React Native	OK	Service HTTP opérationnel, BLE retiré
5 — Tests bout-en-bout	✗	Bloqué par la phase 1
6 — Finition (v1.1.0)	✗	Bloqué par la phase 5

En résumé : la chaîne logicielle (app + protocole + firmware) est complète et le firmware tourne. Ce qui manque, c'est le **câblage physique du moteur, de la LED et du bouton**, qui bloque les tests de bout en bout.

7. Points forts et observations

Points forts

- Architecture **découplée** : le matériel reste autonome, l'app est un compagnon optionnel.
- Repli intelligent **RTC matériel** → **horloge logicielle** : marche même sans DS3231 câblé.
- Protocole HTTP simple et robuste (sondage par `event_id`), compatible **Expo Go** (pas de modules natifs).
- Boucle de rétroaction élégante : le bouton physique enregistre la prise dans l'app.

Points d'attention

- **Identifiants WiFi en clair** dans `config.h` (SSID + mot de passe versionnés dans Git) — à externaliser.
- **Incohérences documentaires** : `rules.md` (Arduino Uno, « pas de connectivité ») contredit la réalité. Le SSID diffère entre les fichiers (`La_Fibre_dOrange_0E96` dans le firmware vs `MG1` affiché dans l'écran Device).
- **HTTP en clair sans authentification** : n'importe qui sur le LAN peut piloter le moteur. Acceptable pour un prototype.
- **Limite du système d'événements** : `event_id` est un `uint8_t` (déborde à 256) et un seul événement est mémorisé. Si deux événements surviennent entre deux sondages, le premier est perdu.